FACULTÉ DES SCIENCES ET TECHNIQUES DE TANGER

Cycle ingénieur LSI

Atelier JEE 3

Application web e-commerce basée sur JSF et JPA

Réalisé par Sara Ouhaddou
Encadrant Pr. Lotfi Elaachak
Module Java EE (JSF, CDI, JPA)
Dépôt `jee-ecommerce-jsf-jpa`
Date Novembre 2026

Résumé

Ce rapport présente la deuxième version d'« Atlas Boutique », l'application de commerce électronique de l'atelier 2, dont la couche web est entièrement réécrite avec **JavaServer Faces 2.2** (Mojarra) et des pages **Facelets**. Les entités JPA, les services, l'injection CDI et la sécurité sont conservés ; les servlets et les pages JSP sont remplacés par six beans CDI, un par « gestion » (connexion, inscription, vitrine, panier, produits, catégories), et par seize pages `.xhtml` bâties sur un gabarit et cinq composants composites.

Le document explique les décisions plutôt que le code : pourquoi garder les services de l'atelier 2, comment le cycle de vie JSF s'accorde avec le motif Post/Redirect/Get, comment concilier les composants JSF avec une politique de sécurité du contenu stricte (`script-src 'self'`), et comment déplacer les règles d'affichage dans Java pour contenter les analyseurs d'expressions EL. Un incident d'exécution (classe `javax.xml.bind.DatatypeConverter` absente sous Java 17) est détaillé. L'ensemble est vérifié par 215 contrôles automatisés et un parcours manuel de quinze scénarios.

Mots-clés : Java EE, JSF, Facelets, CDI, Weld, JPA, EclipseLink, MySQL, Tomcat, sécurité web.

Table des matières

Résumé **1**

1 Introduction **7**

1.1 Contexte et objectif . . . 7
1.2 Décision de périmètre . . . 7
1.3 Ce qui change, ce qui reste . . . 7
1.4 Taille du projet . . . 7
1.5 Plan du rapport . . . 8

2 Conception **9**

2.1 Acteurs et besoins . . . 9
2.2 Modèle du domaine . . . 9
2.3 Règles métier . . . 11
2.4 Héritage . . . 11
2.5 Architecture JSF . . . 11

3 Environnement et construction **14**

3.1 Versions retenues . . . 14
3.2 Corrections apportées aux dépendances du sujet . . . 14
3.3 Configuration de JSF . . . 15
3.4 Construction et démarrage . . . 15
3.5 Compiler pour Java 11 sous Java 17 . . . 15

4 Persistance **17**

4.1 Unité de persistance . . . 17
4.2 Tables obtenues . . . 17
4.3 Verrouillage optimiste . . . 17
4.4 Refus de la base . . . 18

5 Services et injection CDI **19**

5.1 Quatre interfaces . . . 19
5.2 Transactions centralisées . . . 19

5.3 CDI sur Tomcat . . . 19
5.4 Pourquoi cette couche n'a pas bougé . . . 20

6 Couche JSF : beans, portées et navigation 21
6.1 Cycle de vie et conséquences . . . 21
6.2 Choix des portées . . . 21
6.3 Navigation et motif Post/Redirect/Get . . . 22
6.4 Paramètres de vue . . . 22
6.5 Session et authentification . . . 23
6.6 Aides d'affichage dans Java . . . 24

7 Vues Facelets 25
7.1 Gabarit et composition . . . 25
7.2 Composants composites . . . 25
7.3 Composants JSF et politique de sécurité du contenu . . . 26
7.4 Pages réalisées . . . 26
7.4.1 Vitrine et fiche produit . . . 26
7.4.2 Connexion et inscription . . . 26
7.4.3 Panier . . . 26
7.4.4 Administration . . . 30

8 Sécurité 32
8.1 Ce qui est repris, ce qui est adapté . . . 32
8.2 Filtres . . . 32
8.3 Politique de sécurité du contenu . . . 32
8.4 Contrôle d'accès . . . 33

9 Tests 34
9.1 Stratégie . . . 34
9.2 Contrôles automatisés . . . 34
9.3 Tests de la couche web . . . 34
9.4 Analyse statique des vues . . . 35
9.5 Parcours manuel . . . 35

10 Incidents et diagnostics 37
10.1 Erreur 500 après une connexion réussie . . . 37
10.2 Faux positifs d'analyse statique dans Eclipse . . . 38
10.3 Projet de l'atelier précédent dans l'espace de travail . . . 38

10.4 Classes Java 17 illisibles par EclipseLink 38

11 Gestion de version **39**

11.1 Principes 39

11.2 Historique 39

12 Perspectives **40**

13 Conclusion **41**

Bibliographie **42**

A Annexes **43**

A.1 Arborescence du projet 43

A.2 Comptes de démonstration 43

A.3 Diagrammes Mermaid 43

Table des figures

2.1 Diagramme de classes du domaine (inchangé depuis l'atelier 2) 10
2.2 Pages Facelets, beans CDI et services 12
2.3 Structure du projet dans Eclipse : beans, pages et composants 13

3.1 Démarrage de Tomcat : Weld, Mojarra et connexion EclipseLink 16

4.1 Tables créées dans la base `ecommerce_jsf` 17

7.1 Vitrine : cartes de produits et catégories 27
7.2 Recherche et filtre conservés dans l'adresse 27
7.3 Fiche d'un produit 28
7.4 Connexion refusée : message d'erreur 28
7.5 Inscription : erreurs de validation sous les champs 29
7.6 Panier d'un client 29
7.7 Refus d'une quantité supérieure au stock 30
7.8 Administration : liste des produits 31
7.9 Formulaire produit : erreurs de validation 31

8.1 Accès refusé : un client demande une page d'administration 33

9.1 Contrôle en base : lignes du panier après les manipulations 36

A.1 Diagramme Mermaid du modèle métier 44
A.2 Diagramme Mermaid de la couche JSF 44

Liste des tableaux

1.1 Conservé, remplacé, supprimé 8

2.1 Acteurs et fonctionnalités 9
2.2 Règles métier et double protection 11
2.3 Correspondance servlets → beans 11

3.1 Composants et versions 14

6.1 Portées des beans 21

7.1 Composants composites 25

8.1 Menaces et parades 32

9.1 Lots de contrôles 34
9.2 Scénarios du parcours manuel 35

11.1 Séquence de commits 39

A.1 Comptes créés par `SchemaTool` 43

CHAPITRE 1

Introduction

1.1 Contexte et objectif

L'atelier 2 a produit « Atlas Boutique », une application de commerce électronique construite selon le patron MVC2 : des servlets contrôleurs, des vues JSP/JSTL, des services métier et une persistance JPA sur MySQL. L'atelier 3 demande la même application, mais avec une couche web fondée sur **JavaServer Faces** [1] et sur JPA.

Le sujet impose un objectif simple à énoncer : remplacer le couple *servlet + JSP* par le couple *bean + page Facelets*, sans réécrire ce qui n'a aucune raison de changer.

1.2 Décision de périmètre

Deux lectures du sujet étaient possibles.

- **Option A** : conserver les entités, les services, l'injection CDI et la sécurité de l'atelier 2, et ne remplacer que la couche de présentation.
- **Option B** : repartir d'un projet vide et tout réécrire dans le style « JSF pur », y compris l'accès aux données dans les beans.

J'ai retenu l'option A. Les services de l'atelier 2 sont déjà testés, transactionnels et protégés contre les accès concurrents ; les réécrire n'apporterait aucune compétence nouvelle sur JSF et ferait perdre des garanties. La séparation en couches de l'atelier 2 a précisément cet intérêt : changer la technologie d'affichage sans toucher au métier. Ce choix est aussi une validation de l'architecture précédente, ce qui le rend instructif.

1.3 Ce qui change, ce qui reste

Le tableau 1.1 résume la répartition.

1.4 Taille du projet

Le code principal compte 49 classes Java (4 242 lignes), dont 15 pour la nouvelle couche web. Les tests représentent 8 classes (720 lignes). L'interface se compose de 16 fichiers `.xhtml` (933 lignes), d'une feuille de style et d'un fichier JavaScript (439 lignes en tout).

Table 1.1. Conservé, remplacé, supprimé

Statut	Éléments
Conservé tel quel	Entités JPA, exceptions, utilitaires (`PasswordHasher`, `Money`), services et leurs implémentations JPA, `TransactionRunner`, `SchemaTool`, `persistence.xml`, configuration CDI.
Remplacé	Servlets → beans CDI ; pages JSP et *tag files* → pages Facelets et composants composites ; `Flash` → `FacesMessage` ; filtres adaptés au cycle de vie JSF.
Supprimé	jQuery, mini-panier en JSON, `BaseServlet`, `PanierJson`, bibliothèque JSTL.
Nouveau	Beans de gestion, `UserSession`, `Affichage`, tests statiques des vues (`VuesChecks`).

1.5 Plan du rapport

Le chapitre 2 reprend le modèle et présente l'architecture JSF. Le chapitre 3 fixe l'environnement et les versions. Les chapitres 4 et 5 résument les couches conservées. Le cœur de l'atelier se trouve aux chapitres 6 (beans, portées, cycle de vie) et 7 (Facelets). La sécurité (chapitre 8), les tests (chapitre 9) et les incidents (chapitre 10) précèdent la gestion de version, les perspectives et la conclusion.

CHAPITRE 2

Conception

2.1 Acteurs et besoins

Les trois profils de l'atelier 2 sont inchangés (tableau 2.1).

Table 2.1. Acteurs et fonctionnalités

Acteur	Fonctionnalités
Visiteur	Parcourir la vitrine, filtrer par catégorie, rechercher, ouvrir la fiche d'un produit, créer un compte, se connecter.
Client	Tout ce que fait le visiteur, plus : ajouter un produit au panier, modifier une quantité, retirer une ligne, vider le panier.
Administrateur	Créer, modifier, retirer de la vente et supprimer des produits ; gérer les catégories.

2.2 Modèle du domaine

Le modèle est identique à celui de l'atelier 2 : `Client` et `Administrateur` spécialisent la classe abstraite `Internaute` ; un client possède un panier, composé de lignes qui référencent des produits ; la vitrine présente des catégories qui regroupent des produits. La figure 2.1 le représente ; la version Mermaid, lisible directement sur GitHub, figure en annexe (figure A.1).

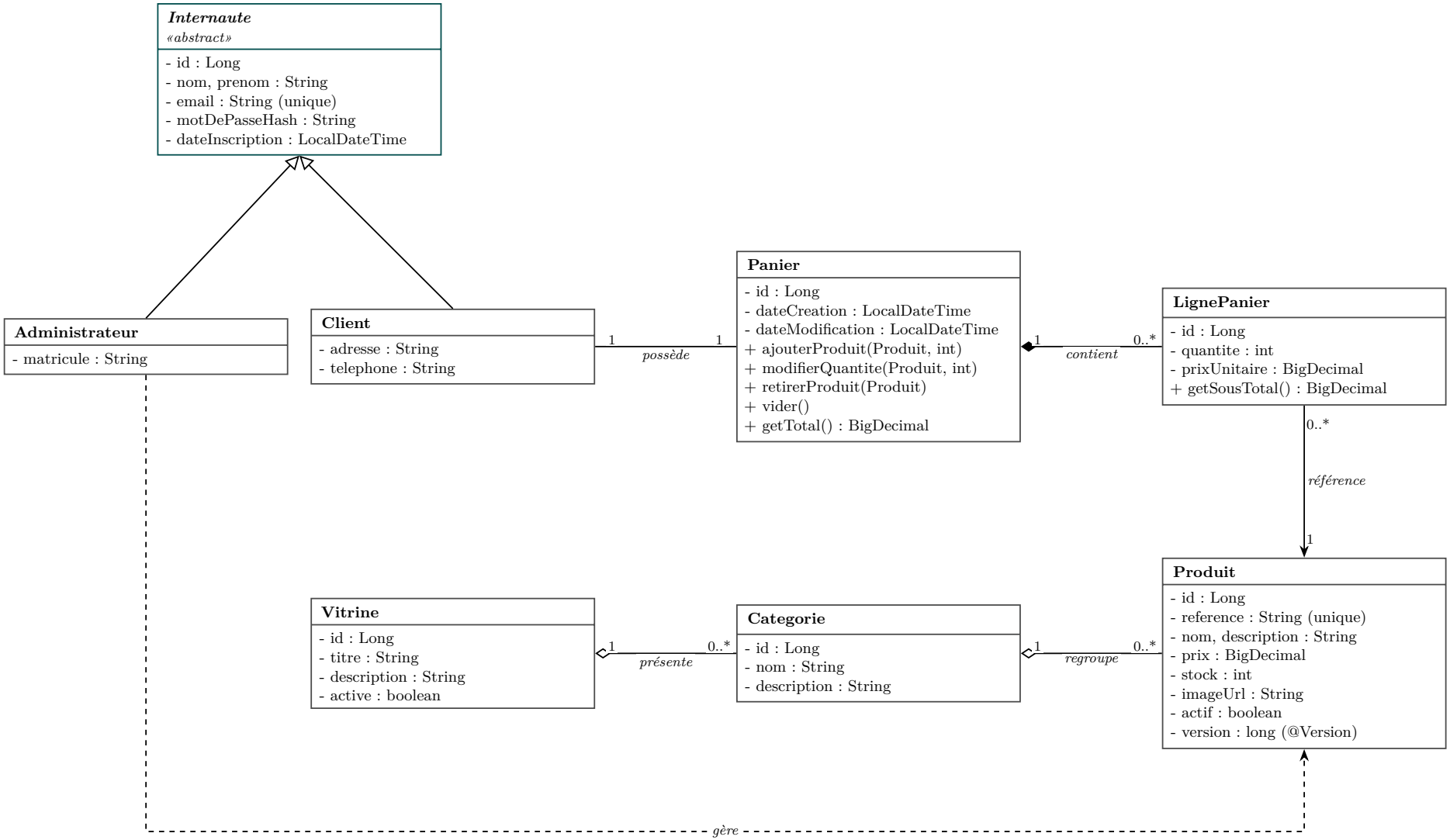


Figure 2.1. Diagramme de classes du domaine (inchangé depuis l'atelier 2)

2.3 Règles métier

Les règles restent portées par l'entité `Panier` et par les contraintes de la base. Aucune n'est dupliquée dans un bean : c'est la raison pour laquelle le remplacement de la couche web n'a demandé aucune modification du métier (tableau 2.2).

Table 2.2. Règles métier et double protection

N°	Règle	Garde-fou en base
R1	Une quantité de ligne est comprise entre 1 et 99.	`CHECK quantite >= 1`
R2	Seul un produit actif peut être ajouté au panier.	—
R3	La quantité d'une ligne ne dépasse pas le stock disponible.	`CHECK stock >= 0`
R4	Ajouter un produit déjà présent fusionne la ligne.	`UNIQUE (panier, produit)`
R5	Le prix de la ligne est figé au moment de l'ajout.	`CHECK prix_unitaire >= 0`
R6	Le total est la somme des sous-totaux, en `BigDecimal`.	—
R7	L'adresse électronique est unique, en minuscules.	`UNIQUE (email)`
R8	La référence d'un produit est unique, en majuscules.	`UNIQUE (reference)`
R9	Un produit présent dans un panier ne peut pas être supprimé.	clé étrangère
R10	Une catégorie qui contient des produits ne peut pas être supprimée.	clé étrangère
R11	Un internaute a obligatoirement un rôle.	`NOT NULL` + `CHECK`

2.4 Héritage

La stratégie `SINGLE_TABLE` est conservée : l'authentification lit un internaute par son adresse sans savoir s'il est client ou administrateur, ce qui se résout en une requête sans jointure. Le détail de la comparaison avec `JOINED` et `TABLE_PER_CLASS` figure dans le rapport de l'atelier 2.

2.5 Architecture JSF

Dans l'atelier 2, une servlet recevait la requête, appelait un service, plaçait des attributs dans la requête puis transmettait à une JSP. Avec JSF, le contrôleur frontal est unique (`FacesServlet`) ; le rôle de chaque servlet est repris par un **bean CDI**, lié à une page par des expressions EL. La figure 2.2 montre les trois niveaux ; le tableau 2.3 donne la correspondance exacte avec l'atelier 2.

Table 2.3. Correspondance servlets → beans

Atelier 2	Atelier 3	Pages
`ClientServlet`	`ConnexionBean`, `InscriptionBean`	`connexion.xhtml`, `inscription.xhtml`
`VitrineServlet`	`VitrineBean`, `DetailBean`	`vitrine.xhtml`, `detail.xhtml`
`PanierServlet`	`PanierBean`	`panier.xhtml`
`ProduitServlet`	`ProduitAdminBean`	`admin/produits.xhtml`, `admin/produit-form.xhtml`
`CategorieServlet`	`CategorieAdminBean`	`admin/categories.xhtml`, `admin/categorie-form.xhtml`

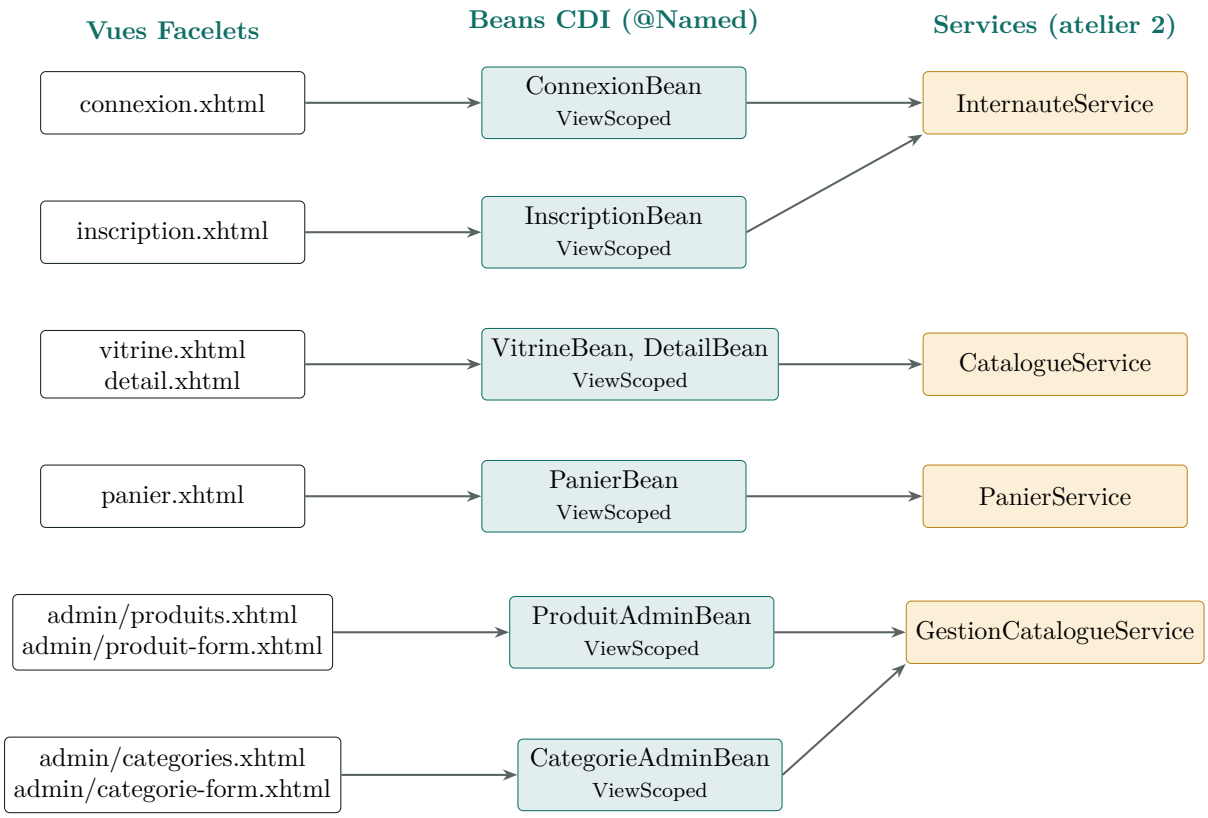


Figure 2.2. Pages Facelets, beans CDI et services

Le sujet suggère un bean par gestion. Je l'ai suivi, avec un découpage plus fin que « un bean par entité » : un bean est lié à un *écran* et à son vocabulaire, pas à une table. Le panier, par exemple, manipule des lignes et des produits mais n'a qu'un bean, parce qu'un seul écran le commande.



Figure 2.3. Structure du projet dans Eclipse : beans, pages et composants

CHAPITRE 3

Environnement et construction

3.1 Versions retenues

Le tableau 3.1 liste les composants et la raison de chaque version. Le sujet fournit une liste de dépendances ; plusieurs entrées ont dû être corrigées pour fonctionner avec le JDK et Tomcat installés.

Table 3.1. Composants et versions

Composant	Version	Remarque
JDK (exécution)	17	Tomcat et Eclipse l'utilisent.
Compilation	`release` 11	EclipseLink 2.7.5 ne lit pas les classes Java 17 (section 3.5).
Tomcat	9.0.123, port 8081	Espace de noms `javax.*`, compatible JSF 2.2.
JSF (Mojarra)	2.2.20	Tomcat ne fournit pas JSF : `jsf-api` et `jsf-impl` sont embarqués dans le WAR.
CDI (Weld)	3.1.9.Final	Version *servlet shaded* pour Tomcat.
JPA (EclipseLink)	2.7.5	Apporte l'API `javax.persistence` 2.2.
Pilote MySQL	8.0.33	Remplace le 8.0.12 du sujet, compatible MySQL 8.
Servlet API	4.0.1, `provided`	Fournie par Tomcat ; ne doit pas se trouver dans le WAR.
JAXB API	2.3.1	Absente du JDK depuis Java 11 ; requise par Mojarra (chapitre 10).

3.2 Corrections apportées aux dépendances du sujet

— **Servlet API** : le sujet donne la version 4.0.0 sans portée. Sans `provided`, l'API se retrouve dans le WAR et entre en conflit avec celle de Tomcat.

— **JPA** : l'artefact `org.eclipse.persistence:javax.persistence:2.0.0` du sujet n'est *pas* déclaré. EclipseLink 2.7.5 apporte déjà l'API 2.2 ; deux versions de l'API sur le même chemin de classes provoqueraient des conflits.

— **JSTL et JSP** : Facelets n'en a pas besoin (les balises `c:` et `fn:` sont implémentées dans Mojarra). Les déclarations de l'atelier 2 sont donc retirées.

— **JAXB** : ajouté après l'incident du chapitre 10.

Listing 3.1. Extrait du `pom.xml` : JSF et JAXB

```
<dependency>
  <groupId>com.sun.faces</groupId>
  <artifactId>jsf-api</artifactId>
  <version>${mojarra.version}</version>
</dependency>
<dependency>
```

```
  <groupId>com.sun.faces</groupId>
  <artifactId>jsf-impl</artifactId>
  <version>${mojarra.version}</version>
</dependency>

<!--
  JAXB API. Mojarra 2.2.x encrypts the "flash" cookie (it carries the message kept across a
  redirect, e.g. "Bienvenue" after sign-in) with javax.xml.bind.DatatypeConverter, a class
  removed from the JDK in Java 11. Tomcat runs on Java 17 here, so without this dependency
  every redirect that keeps a message fails with ClassNotFoundException (HTTP 500).
-->
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>
```

3.3 Configuration de JSF

Le contrôleur frontal est déclaré dans `web.xml` et associé au motif `*.xhtml`. Ce choix garde des adresses lisibles (`vitrine.xhtml?categorie=2`) et signifie qu'une page Facelets est atteinte directement par son nom, sans préfixe `/faces/`.

Listing 3.2. Déclaration de `FacesServlet`

```
<servlet>
  <servlet-name>FacesServlet</servlet-name>
  <servlet-class>javax.faces.webapp.FacesServlet</servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>FacesServlet</servlet-name>
  <url-pattern>*.xhtml</url-pattern>
</servlet-mapping>
```

Le même fichier configure la phase de développement (`PROJECT_STAGE=Development`, messages d'erreur plus détaillés), la sauvegarde de l'état de la vue sur le serveur, l'ordre des filtres (sécurité puis authentification), les pages d'erreur (403, 404, 405, 500 et toute exception) et un bloc de sécurité interdisant l'accès direct à `/resources/components/*`.

3.4 Construction et démarrage

La construction se fait par Maven (`mvn -q clean package`), le déploiement par Eclipse sur Tomcat. Au démarrage, la console doit montrer, dans l'ordre : l'initialisation de Weld, celle de Mojarra, puis la connexion EclipseLink (`login successful`). La figure 3.1 en donne la trace.

3.5 Compiler pour Java 11 sous Java 17

EclipseLink 2.7.5 lit les classes des entités avec une version d'ASM qui rejette les fichiers de classe plus récents que Java 13 (« Unsupported class file major version 61 »). Compilées pour Java 17, les entités ne seraient pas découvertes et l'application démarrerait sans table. La propriété `maven.compiler.release=11` compile pour Java 11 tout en gardant le JDK 17 : les classes ainsi

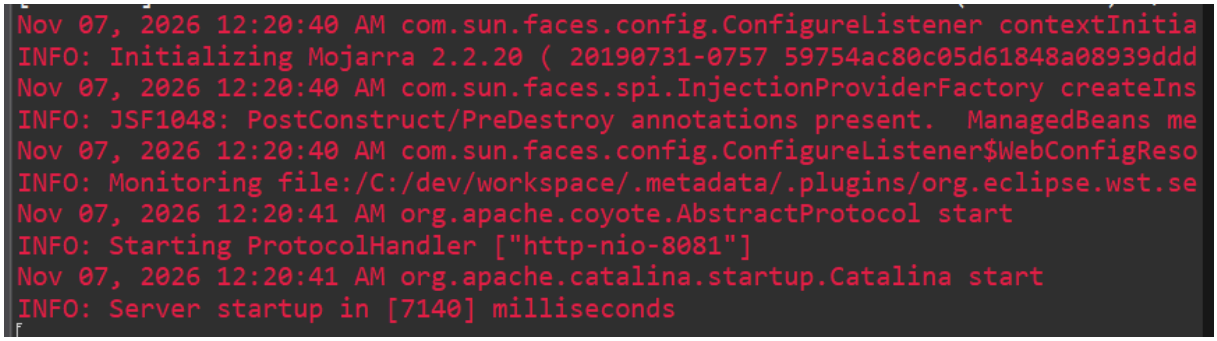


Figure 3.1. Démarrage de Tomcat : Weld, Mojarra et connexion EclipseLink

produites s'exécutent normalement sous Java 17. Cette contrainte était déjà apparue à l'atelier 2 ; elle est ici reconduite et documentée dans le `pom.xml`.

CHAPITRE 4

Persistance

La couche de persistance est reprise sans modification de l'atelier 2. Ce chapitre en rappelle les décisions, car elles conditionnent le comportement des beans.

4.1 Unité de persistance

Le fichier `persistence.xml` déclare une unité en `RESOURCE_LOCAL`, liste explicitement les huit entités et fixe `jpa.ddl=none` : EclipseLink ne crée jamais les tables lui-même. Le schéma est produit par un outil dédié, `SchemaTool` (mode `RESET`), qui génère les tables puis ajoute les contraintes que le mappage seul ne sait pas exprimer (`CHECK`, unicité composite, colonne discriminante non nulle).

4.2 Tables obtenues

Six tables sont créées. La figure 4.1 montre le résultat dans MySQL après `SchemaTool`.

```
PS C:\dev\ecommerce-jsf-jpa> mysql -u root -p ecommerce_jsf -e "
SHOW TABLES;"
Enter password: ****************
+------------------------+
| Tables_in_ecommerce_jsf |
+------------------------+
| categorie              |
| internaute             |
| ligne_panier           |
| panier                 |
| produit                |
| vitrine                |
+------------------------+
```

Figure 4.1. Tables créées dans la base `ecommerce_jsf`

4.3 Verrouillage optimiste

`Produit` porte une colonne `@Version`. Si deux clients ajoutent au panier le dernier exemplaire en même temps, le second voit sa transaction refusée plutôt que d'écraser la première. Le bean n'a rien à faire : l'exception est traduite en message métier par la couche service.

4.4 Refus de la base

Les contraintes de la base interceptent ce que la couche métier aurait oublié. Elles sont traduites en messages lisibles par la classe `SqlErrors`, de sorte qu'un bean n'affiche jamais un message SQL brut.

CHAPITRE 5

Services et injection CDI

5.1 Quatre interfaces

La couche service expose quatre interfaces : `InternauteService` (inscription, authentification), `CatalogueService` (lecture : vitrine, catégories, recherche paginée, fiche produit), `PanierService` (opérations sur le panier) et `GestionCatalogueService` (administration). Les beans ne connaissent que ces interfaces ; les classes `Jpa*Service` sont des implémentations interchangeables.

5.2 Transactions centralisées

Toute écriture passe par `TransactionRunner`. Il ouvre un `EntityManager`, démarre la transaction, exécute l'opération, valide ou annule, puis ferme. Aucun bean, aucun service ne manipule une transaction à la main. L'extrait ci-dessous montre le principe : une seule porte d'entrée, un seul endroit où une transaction peut rester ouverte par erreur.

Listing 5.1. Écriture transactionnelle

```
    /** Read-write work: commit when it returns, rollback when it throws. */
    public <T> T write(Function<EntityManager, T> work) {
        EntityManager em = emf.createEntityManager();
        EntityTransaction tx = em.getTransaction();
        try {
            tx.begin();
            T result = work.apply(em);
            tx.commit();
            return result;
        } catch (RuntimeException e) {
            if (tx.isActive()) {
                tx.rollback();
            }
            throw SqlErrors.traduire(e);
        } finally {
            em.close();
        }
    }
}
```

5.3 CDI sur Tomcat

Tomcat n'est pas un serveur d'applications complet : il ne fournit pas CDI. Weld est donc embarqué, et trois éléments le branchent : `beans.xml` (mode « annoté »), une ressource `BeanManager` dans `META-INF/context.xml`, et la référence correspondante dans `web.xml`. Une

fabrique (`EntityManagerFactoryProducer`) fournit l'`EntityManagerFactory` unique, créé à la première demande.

Listing 5.2. Production de l'`EntityManagerFactory`

```
@ApplicationScoped
public class EntityManagerFactoryProducer {

    @Produces
    @ApplicationScoped
    public EntityManagerFactory creer() {
        return PersistenceConfig.createEntityManagerFactory();
    }

    public void fermer(@Disposes EntityManagerFactory emf) {
        if (emf.isOpen()) {
            emf.close();
        }
    }

    /**
     * Forces the creation at startup: a wrong password or an unreachable MySQL
     * shows up in the Tomcat log when the application starts, not at the
     * first visitor's request.
     */
    void demarrer(@Observes @Initialized(ApplicationScoped.class) Object evenement,
                  EntityManagerFactory emf) {
        emf.isOpen();
    }
}
```

5.4 Pourquoi cette couche n'a pas bougé

Les services ne dépendent ni de `HttpServletRequest`, ni de JSF. C'est ce qui a rendu l'option A possible : changer la technologie web n'a demandé aucune modification d'une classe de `ma.fstt.service` ou de `ma.fstt.jpa`.

CHAPITRE 6

Couche JSF : beans, portées et navigation

6.1 Cycle de vie et conséquences

Une requête JSF traverse six phases : restauration de la vue, application des valeurs de requête, validation, mise à jour du modèle, invocation de l'application, rendu. Deux conséquences structurent tout le code.

— Une valeur saisie n'arrive dans le bean qu'à la phase de *mise à jour du modèle*, et seulement si la validation a réussi. Le bean ne lit donc jamais `request.getParameter` : il reçoit des propriétés.

— Une méthode d'action (`action="#{bean.methode}"`) s'exécute à la phase d'*invocation de l'application*, après la mise à jour du modèle. Elle retourne un résultat de navigation.

6.2 Choix des portées

La portée d'un bean décide de la durée de vie de son état. Le tableau 6.1 justifie chaque choix.

Table 6.1. Portées des beans

Bean	Portée	Justification
`ConnexionBean`, `InscriptionBean`	`@ViewScoped`	Garde la saisie tant que la même page est réaffichée (erreur de validation) ; disparaît à la navigation.
`VitrineBean`, `DetailBean`	`@ViewScoped`	Les filtres arrivent par l'adresse (paramètres de vue) ; l'état n'a pas à survivre à la page.
`PanierBean`	`@ViewScoped`	Le panier lui-même est lu en base à chaque affichage ; le bean ne garde que les quantités saisies.
`ProduitAdminBean`, `CategorieAdminBean`	`@ViewScoped`	Un formulaire d'édition par page.
`UserSession`	`@RequestScoped`	Façade sans état propre : l'état de l'utilisateur est dans la `HttpSession`, sous forme d'un objet `SessionUser` sérialisable.
`LoginGuard`, `Affichage`	`@ApplicationScoped`	Partagés : le compteur de tentatives doit voir tous les visiteurs, et les aides d'affichage n'ont pas d'état.

Deux décisions méritent d'être détaillées.

Aucun bean en `@SessionScoped`. Une portée session est tentante pour « se souvenir de l'utilisateur », mais elle fait vivre l'état d'un bean aussi longtemps que la session. Ici, la session ne contient qu'un petit objet immuable (`SessionUser` : identifiant, nom, rôle) ; `UserSession` est une façade de requête qui le lit et le modifie. La fixation de session se traite ainsi à un seul endroit (section 6.5).

`@ViewScoped` de JSF, pas de CDI. L'annotation utilisée est `javax.faces.view.ViewScoped`, couplée à `@Named`. Les beans de cette portée sont placés dans l'état de la vue : ils doivent être `Serializable`, ce qui a conduit à rendre sérialisables les objets formulaires (`ProduitForm`, `CategorieForm`, `InscriptionForm`) et à remplacer les `Map` génériques par des `LinkedHashMap`.

6.3 Navigation et motif Post/Redirect/Get

Une action qui modifie des données doit se terminer par une redirection, sinon un rechargement de la page (`F5`) rejoue le formulaire [13]. JSF y répond par la navigation implicite : une méthode retourne le nom d'une page suivi de `?faces-redirect=true`. Les paramètres utiles (`includeViewParams=true`) sont conservés dans l'adresse.

Le message de confirmation pose un problème : il est créé pendant la requête qui poste, mais affiché dans la page suivante. La solution est `FacesMessage` conservé par le *flash* de JSF. La classe utilitaire `Messages` regroupe les trois niveaux (succès, information, erreur) et la méthode `apresRedirection()` qui active la conservation.

Listing 6.1. `PanierBean` : toute action passe par une méthode unique

```
private String executer(String cible, String messageSucces, Operation operation) {
    try {
        Panier resultat = operation.appliquer();
        userSession.memoriserCompteur(resultat.getNombreArticles());
        Messages.succes(messageSucces);
    } catch (MetierException e) {
        Messages.erreur(e.getMessage());
    }
    panier = null;
    quantites = null;
    Messages.apresRedirection();
    return cible;
}
```

La méthode `executer` évite de répéter quatre fois le même bloc. Elle capture l'exception métier, la transforme en message rouge, vide les valeurs mises en cache (`panier = null`) pour que l'écran suivant relise la base, active la conservation du message, et retourne la cible. Le motif est le même dans tous les beans de gestion.

6.4 Paramètres de vue

La vitrine se filtre par catégorie, mot-clé et page. Ces trois valeurs sont portées par l'adresse, pas par un formulaire JSF : la recherche est un simple formulaire `GET`. Les paramètres sont déclarés dans la page avec `f:viewParam`, et `f:viewAction` appelle `init()` une fois qu'ils sont injectés.

Listing 6.2. Métadonnées de `vitrine.xhtml`

```
<f:metadata>
  <f:viewParam name="categorie" value="#{vitrineBean.categorie}"/>
  <f:viewParam name="q" value="#{vitrineBean.q}"/>
```

```
  <f:viewParam name="page" value="#{vitrineBean.page}"/>
  <f:viewParam name="deconnecte" value="#{vitrineBean.deconnecte}"/>
  <f:viewAction action="#{vitrineBean.init}"/>
</f:metadata>
```

L'avantage est concret : une recherche se partage et se met en favori (`vitrine.xhtml?categorie=2&q=cle`), le bouton « page précédente » du navigateur fonctionne, et la pagination se construit avec de simples liens. La construction de ces adresses est un calcul pur (`VitrineBean.construireUrl`), vérifié par des tests sans serveur : les paramètres sont encodés en UTF-8, la page 1 est omise et un `&` saisi dans la recherche ne peut pas injecter un paramètre supplémentaire.

6.5 Session et authentification

`UserSession.ouvrir` est le seul endroit où une connexion réussie est enregistrée. Il enchaîne quatre opérations dans cet ordre précis.

Listing 6.3. Ouverture de session

```
public String ouvrir(Internaute internaute) {
    HttpServletRequest req = (HttpServletRequest) contexte().getRequest();
    HttpSession ancienne = req.getSession(true);
    String retour = (String) ancienne.getAttribute(AuthFilter.RETOUR);
    req.changeSessionId();

    HttpSession session = req.getSession();
    session.removeAttribute(AuthFilter.RETOUR);
    SessionUser user = SessionUser.de(internaute);
    session.setAttribute(SessionUser.CLE, user);
    SecurityFilter.nouveauJeton(session);
    session.setAttribute(COMPTEUR,
            user.isClient() ? paniers.consulter(user.getId()).getNombreArticles() : 0);

    String cible;
    if (user.isAdmin()) {
        cible = "/admin/produits.xhtml";
    } else {
        cible = Chemins.cheminSur(retour, "/vitrine.xhtml");
        if (cible.startsWith("/admin")) {
            cible = "/vitrine.xhtml";
        }
    }
    return Chemins.avecParametre(cible, "faces-redirect", "true");
}
```

1. Lire l'adresse de retour mémorisée par le filtre d'authentification, avant de changer d'identifiant.
2. `changeSessionId()` : un nouvel identifiant de session est émis (défense contre la fixation de session).
3. Enregistrer le `SessionUser` dans la nouvelle session.
4. Renouveler le jeton anti-CSRF, lié à l'identité qui vient de s'authentifier.

`ConnexionBean.connecter` appelle cette méthode après avoir interrogé le service et le `LoginGuard`, qui limite les essais à cinq par quinze minutes pour un couple adresse IP / adresse

électronique. Le mot de passe saisi est vidé dans tous les cas d'échec, pour qu'il ne soit jamais renvoyé à la page.

Listing 6.4. `ConnexionBean.connecter`

```
public String connecter() {
    String adresseEmail = email == null ? "" : email.trim();
    HttpServletRequest req = (HttpServletRequest) FacesContext.getCurrentInstance()
            .getExternalContext().getRequest();
    String cle = req.getRemoteAddr() + "|" + adresseEmail.toLowerCase(Locale.ROOT);

    long attente = guard.secondesRestantes(cle);
    if (attente > 0) {
        Messages.erreur("Trop de tentatives. Réessaie dans " + ((attente + 59) / 60) + "
    minute(s).");
        motDePasse = null;
        return null;
    }
    try {
        Internaute internaute = internautes.authentifier(adresseEmail, motDePasse == null ? ""
     : motDePasse);
        guard.reussite(cle);
        String cible = userSession.ouvrir(internaute);
        Messages.succes("Bienvenue " + internaute.getPrenom() + " !");
        Messages.apresRedirection();
        return cible;
    } catch (MetierException e) {
        guard.echec(cle);
        Messages.erreur(e.getMessage());
        motDePasse = null; // the password is never sent back to the page
        return null;
    }
}
```

6.6 Aides d'affichage dans Java

Le bean `Affichage` (portée application) fournit le format des prix (deux décimales, virgule, espace insécable, MAD), les initiales d'un nom, les classes CSS de stock, le libellé du bouton d'ajout et les textes de la page d'erreur. Ces règles auraient pu s'écrire en EL dans les pages. Elles sont en Java pour une raison pratique expliquée au chapitre 10 : les analyseurs statiques d'Eclipse signalent comme erreurs des expressions conditionnelles parfaitement valides à l'exécution. Déplacer la logique dans Java supprime ces faux positifs et la rend testable (section 9.3).

CHAPITRE 7

Vues Facelets

7.1 Gabarit et composition

Facelets remplace JSP comme technologie de vue : les pages sont des documents XHTML bien formés, ce qui permet de les valider avant de lancer le serveur. Le gabarit `WEB-INF/templates/layout.xhtml` contient le squelette commun (en-tête, menu latéral, zone de messages, pied de page) et trois points d'insertion : les métadonnées, le contenu et, au besoin, des scripts. Chaque page déclare le gabarit avec `template=` et remplit ses zones avec `ui:define`. Le gabarit est placé sous `WEB-INF` : aucune adresse ne permet de le demander directement.

Les messages JSF s'affichent à un seul endroit, dans le gabarit, avec une boucle sur `facesContext.messageList`. La classe CSS de chaque message dépend de sa gravité.

Listing 7.1. Affichage des messages dans le gabarit

```
<ui:repeat value="#{facesContext.messageList}" var="message">
  <div class="flash flash-#{message.severity.ordinal == 2 ? 'error' : (message.severity.
    ordinal == 0 ? 'success' : 'info')}"
       role="status">#{message.summary}</div>
</ui:repeat>
```

7.2 Composants composites

Cinq fragments répétés dans plusieurs pages sont devenus des composants composites, rangés dans `resources/components` (espace de noms `http://xmlns.jcp.org/jsf/composite/components`, préfixe `t`).

Table 7.1. Composants composites

Composant	Rôle
`t:csrf`	Champ caché portant le jeton anti-CSRF ; présent dans chaque formulaire.
`t:icone`	Icône vectorielle, dessinée à partir d'un tracé fourni par `Affichage.icone`.
`t:visuel`	Image d'un produit, ou pastille d'initiales quand il n'a pas d'image.
`t:vide`	Bloc « rien à afficher » avec illustration, titre, texte et action facultative.
`t:champ`	Champ de formulaire (texte, mot de passe ou zone de texte) avec son libellé et l'erreur correspondante, lue dans la table d'erreurs du service.

Un composant composite n'a pas de classe Java ; c'est un modèle paramétré. Sa valeur est de rendre impossible un oubli : tout formulaire inclut `t:csrf`, et un test statique le vérifie (chapitre 9). Comme ces fichiers ne sont pas des pages, le `web.xml` interdit leur accès direct.

Le composant `t:champ` traduit un choix de conception : la validation JSF (`required`, validateurs)

n'est pas utilisée. Les règles de saisie vivent dans la couche service, comme à l'atelier 2, et renvoient une table « champ → message ». Une seule source de validation évite que la page et le métier divergent.

7.3 Composants JSF et politique de sécurité du contenu

La politique `script-src 'self'` (chapitre 8) interdit tout script en ligne. Or plusieurs composants JSF classiques en génèrent : `h:commandLink` produit un lien dont le `onclick` appelle une fonction JavaScript, et `f:ajax` injecte du code. Le choix est donc le suivant.

— Les actions sont déclenchées par `h:commandButton`, un vrai bouton d'envoi qui n'a besoin d'aucun script.

— La navigation simple utilise des liens HTML ordinaires, dont l'adresse est calculée par le bean.

— La confirmation avant suppression passe par un attribut `data-confirm` lu par `app.js`, un fichier servi par le serveur lui-même.

— La recherche est un formulaire `GET` en HTML pur, pas un formulaire JSF.

Cette contrainte coûte quelques lignes en moins de confort, et apporte une garantie : aucune injection de balise `<script>` ne peut s'exécuter, même si un échappement venait à être oublié. Elle est vérifiée par `VuesChecks`, qui refuse `h:commandLink`, `f:ajax` et tout attribut `on*`.

7.4 Pages réalisées

Seize fichiers `.xhtml` composent l'interface. Les figures qui suivent montrent le parcours d'un visiteur, d'un client, puis d'un administrateur.

7.4.1 Vitrine et fiche produit

La vitrine affiche des cartes de produits, des pastilles de catégories avec leur effectif, un compteur et une pagination. Selon le profil, le bas de chaque carte propose « Ajouter au panier » (client), « Se connecter pour acheter » (visiteur), ou rien (administrateur).

7.4.2 Connexion et inscription

En cas d'échec de connexion, un message rouge s'affiche et le champ du mot de passe est vidé. Le formulaire d'inscription valide tous les champs en une passe et affiche une erreur sous chacun, en plus d'un message général.

7.4.3 Panier

Le panier liste les lignes avec leur sous-total, un champ de quantité et le total en dirhams. Lorsqu'une quantité dépasse le stock, le service lève une exception métier ; le bean la transforme en message et le panier reste inchangé.

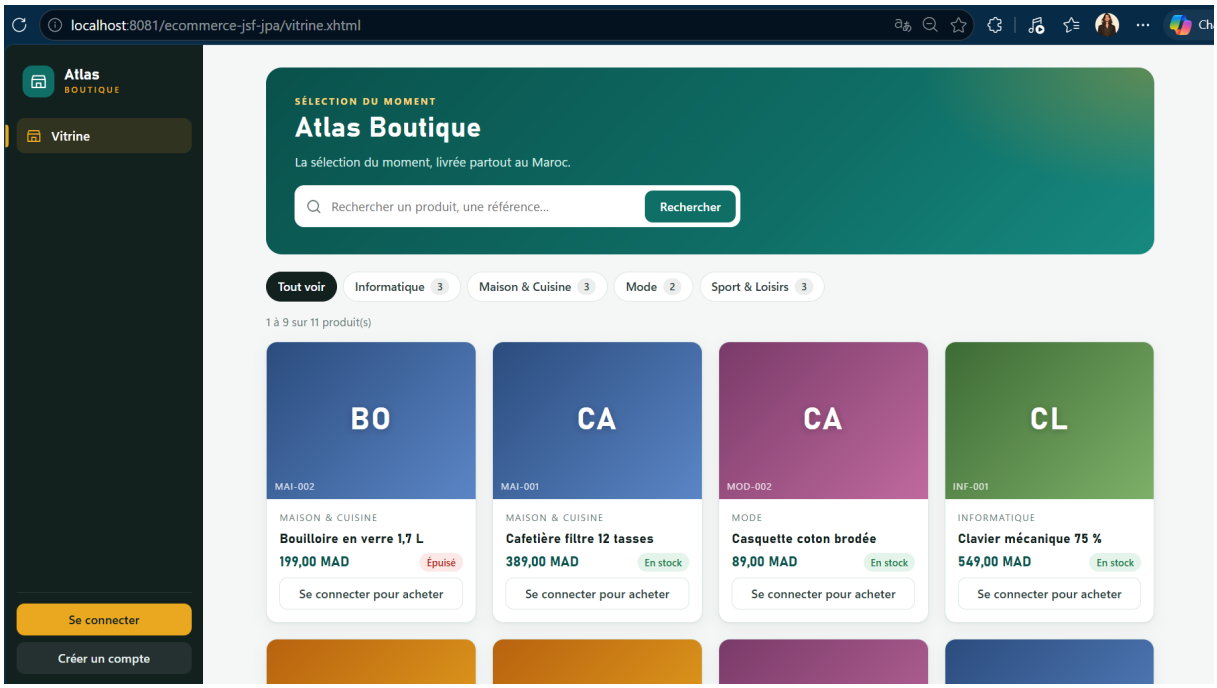


Figure 7.1. Vitrine : cartes de produits et catégories

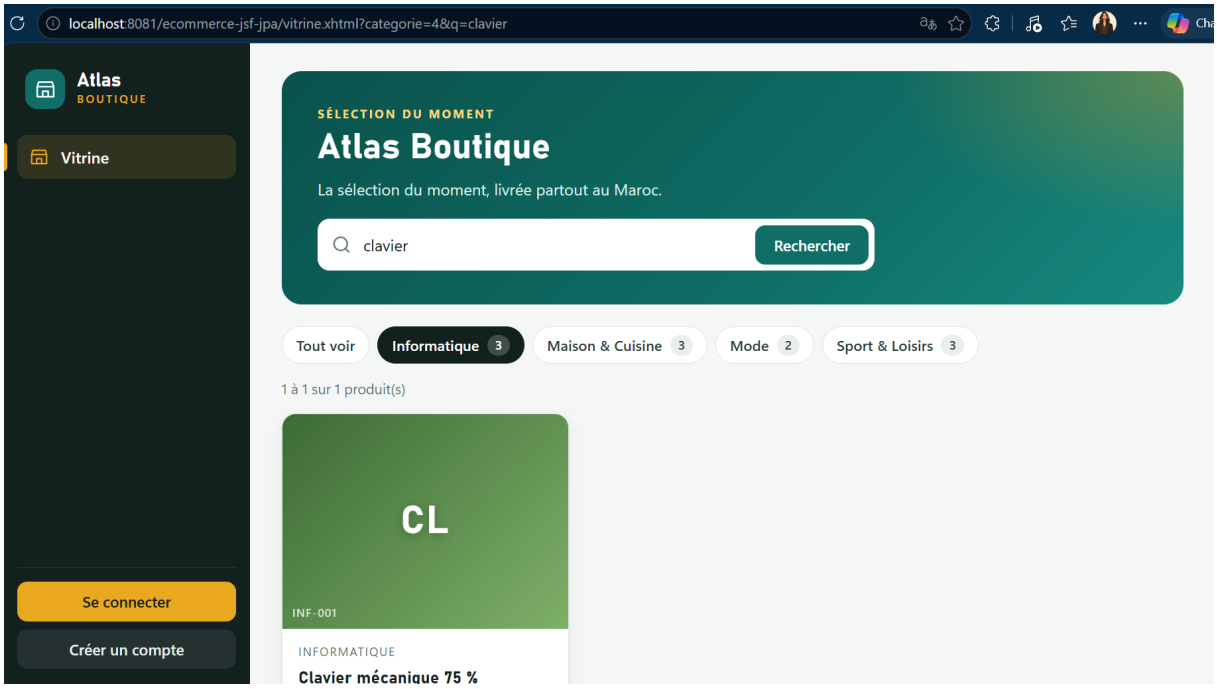


Figure 7.2. Recherche et filtre conservés dans l'adresse

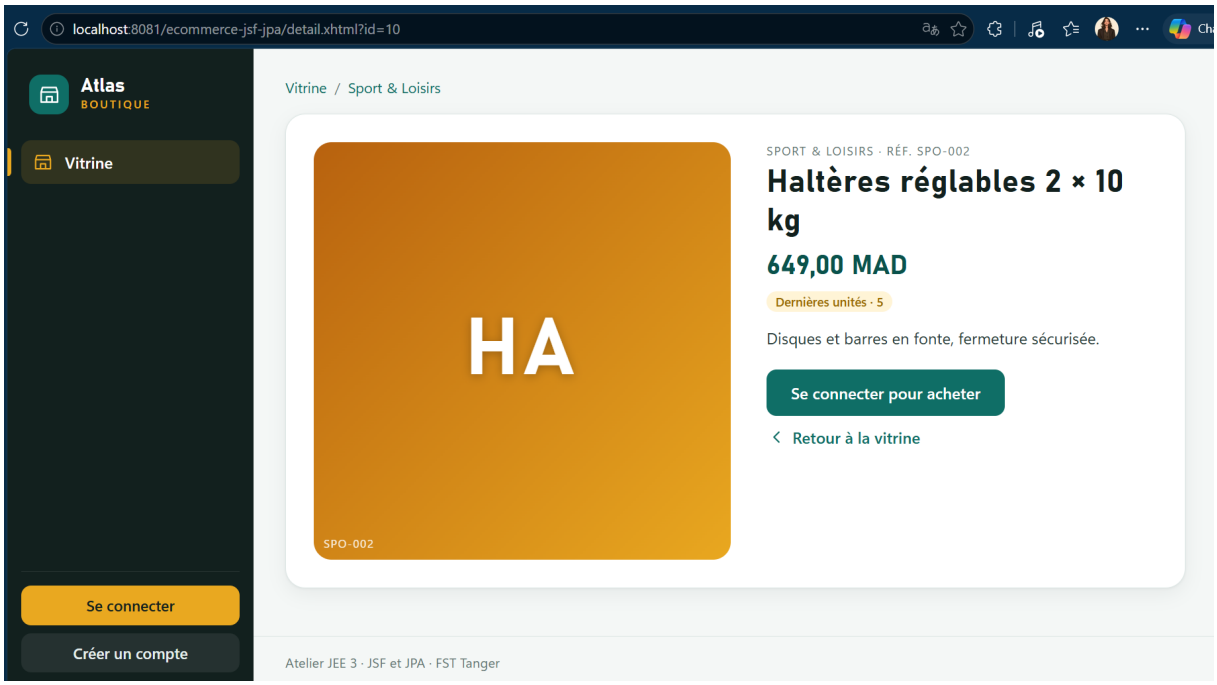


Figure 7.3. Fiche d'un produit

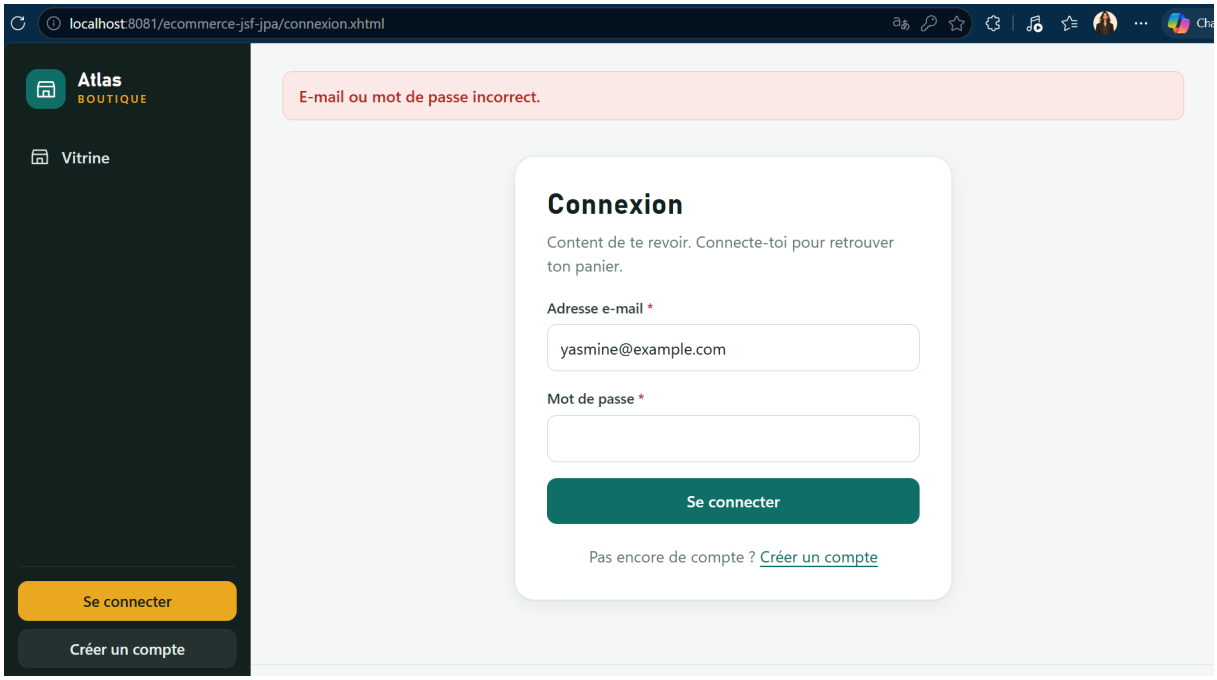


Figure 7.4. Connexion refusée : message d'erreur

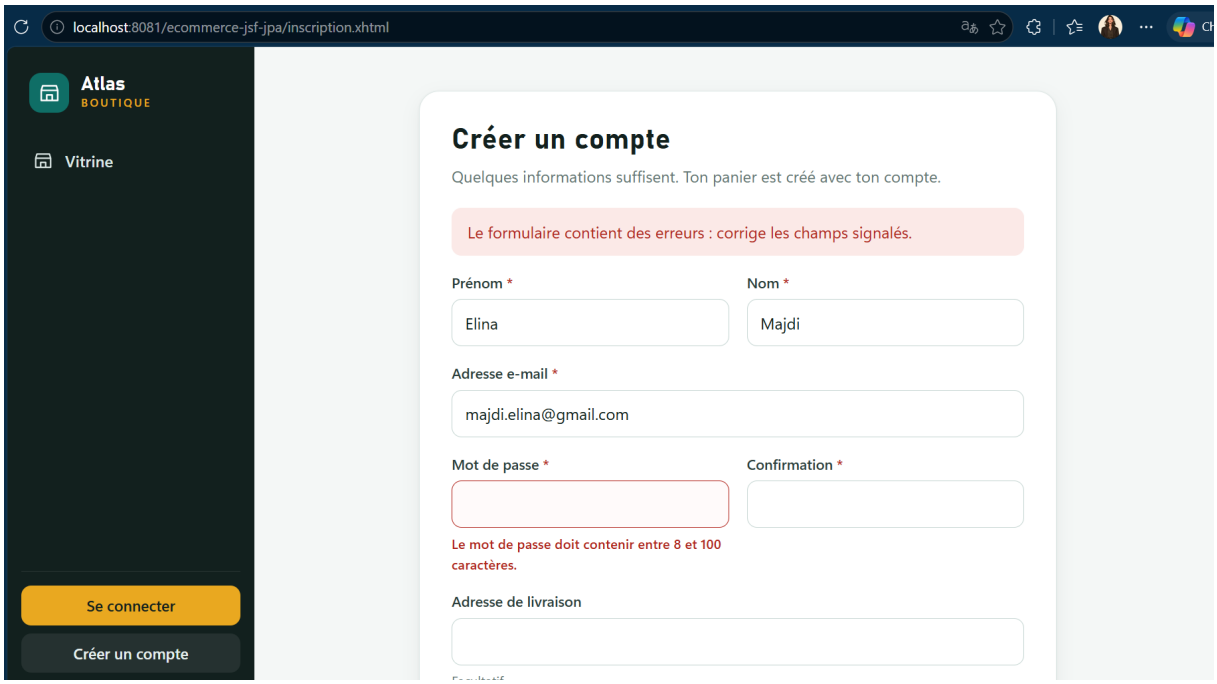


Figure 7.5. Inscription : erreurs de validation sous les champs

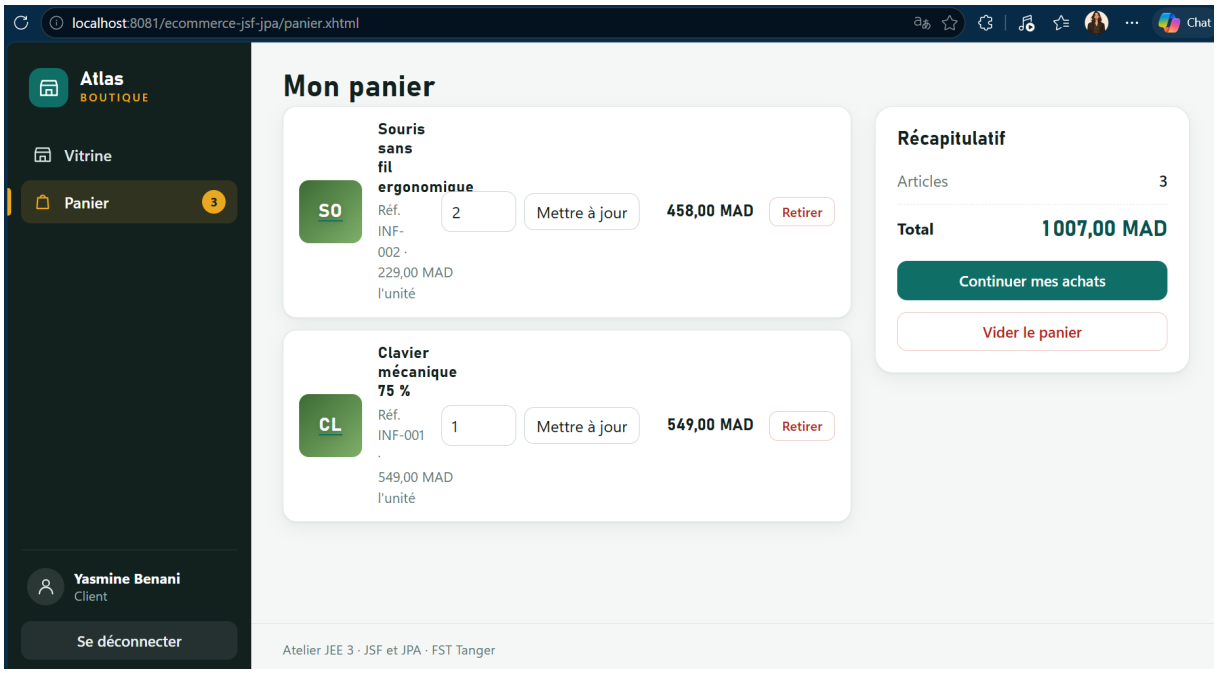


Figure 7.6. Panier d'un client

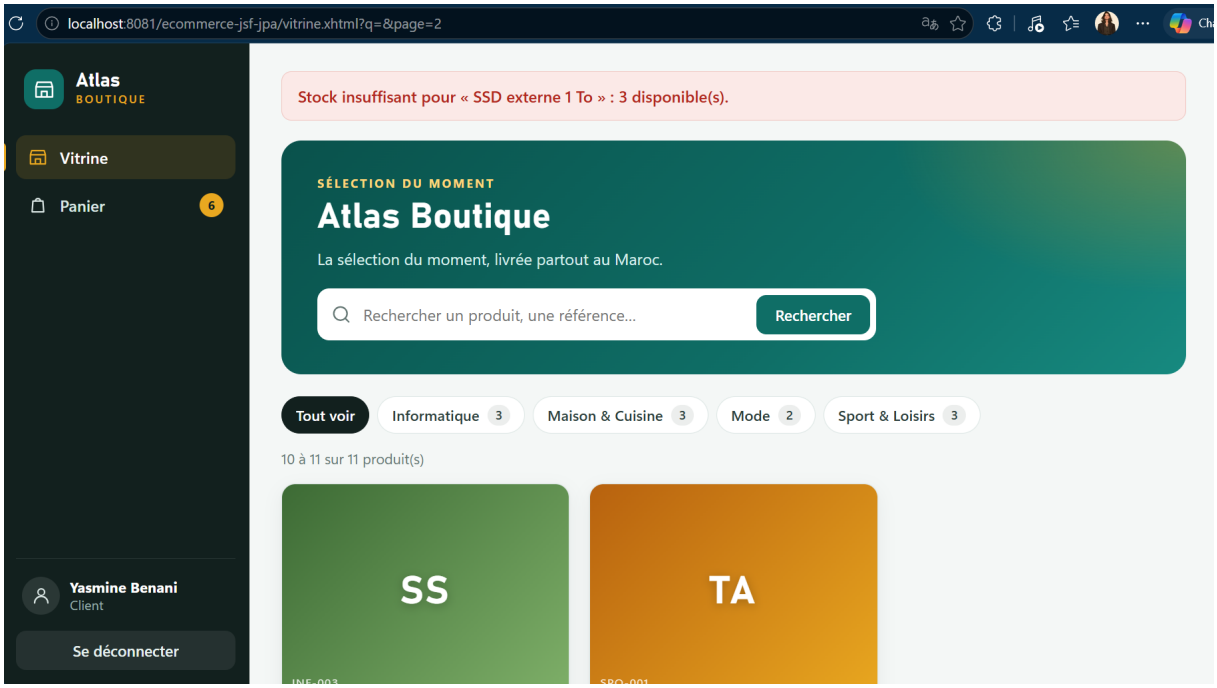


Figure 7.7. Refus d'une quantité supérieure au stock

7.4.4 Administration

Les écrans d'administration sont réservés au rôle `ADMIN` : liste des produits avec actions (modifier, retirer de la vente, supprimer avec confirmation), formulaire produit, gestion des catégories.

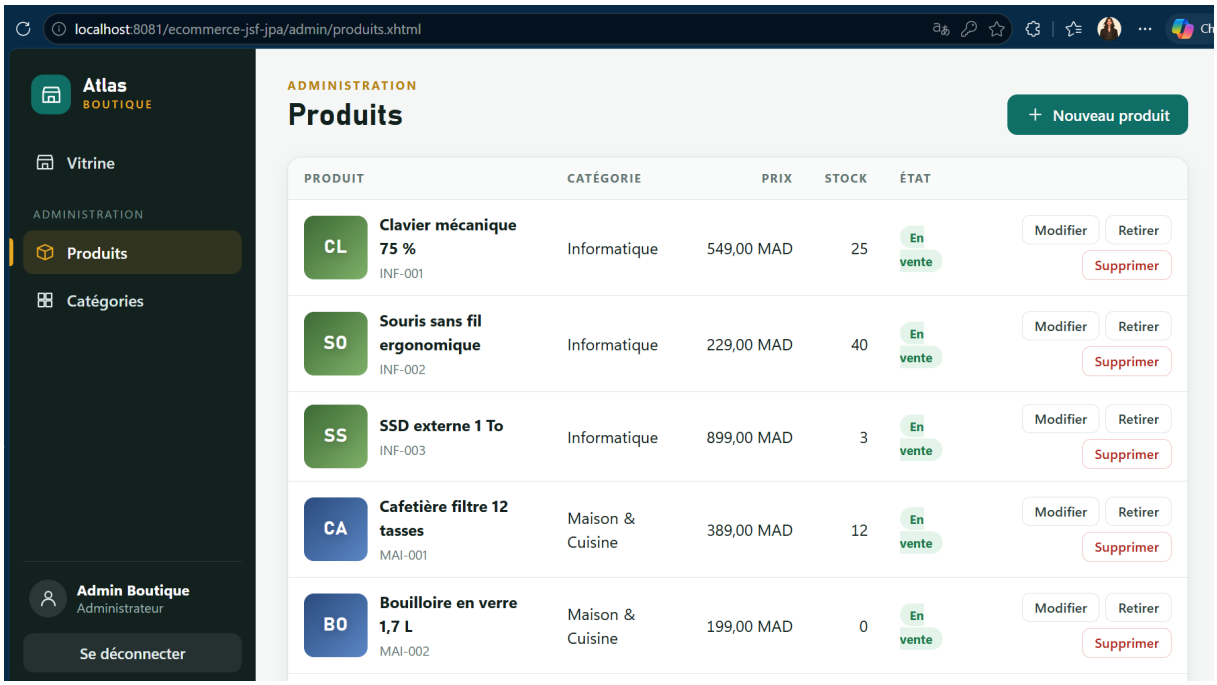


Figure 7.8. Administration : liste des produits

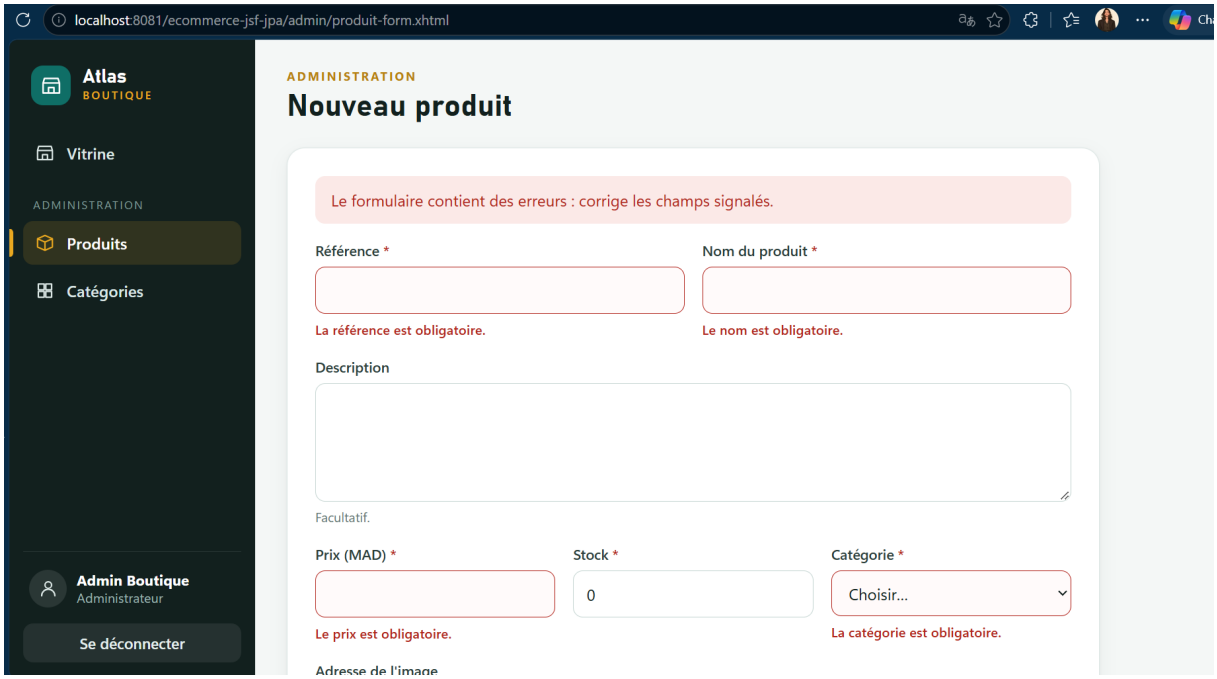


Figure 7.9. Formulaire produit : erreurs de validation

CHAPITRE 8

Sécurité

8.1 Ce qui est repris, ce qui est adapté

Les mécanismes de l'atelier 2 sont conservés ; leur emplacement change, parce que les servlets ont disparu. Le tableau 8.1 relie chaque menace à sa parade dans la version JSF.

Table 8.1. Menaces et parades

Menace	Parade
Injection SQL	Requêtes JPQL paramétrées, aucun assemblage de texte (inchangé).
XSS	Facelets échappe par défaut toute expression `#{...}` ; aucune sortie en `escape="false"` ; CSP `script-src 'self'`.
CSRF	Jeton `_csrf` par session, inséré par `t:csrf`, vérifié en temps constant par `SecurityFilter` sur tout POST. À cela s'ajoute le `ViewState` de JSF.
Fixation de session	`changeSessionId()` et renouvellement du jeton à la connexion (`UserSession.ouvrir`).
Force brute	`LoginGuard` : 5 essais par 15 minutes, par couple IP / adresse.
Redirection ouverte	`Chemins.cheminSur` n'accepte que les chemins internes (refus de `//`, d'URL absolue, d'antislash, de saut de ligne).
Contrôle d'accès	`AuthFilter` par motif d'adresse : `/admin/*` exige le rôle `ADMIN`, `/panier.xhtml` le rôle `CLIENT` (réponse 403 sinon).
Mots de passe	PBKDF2 avec sel aléatoire ; jamais renvoyés à la page.
Accès aux composants	Contrainte de sécurité sur `/resources/components/*`.

8.2 Filtres

Deux filtres s'appliquent, dans cet ordre déclaré dans `web.xml`.

`SecurityFilter` (toutes les adresses) : fixe l'encodage UTF-8, ajoute les en-têtes de sécurité (`Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`) et `Cache-Control: no-store`, génère le jeton CSRF à la première visite et refuse tout `POST` dont le jeton est absent ou différent.

`AuthFilter` (`/panier.xhtml`, `/admin/*`) : sans utilisateur en session, redirige vers `connexion.xhtml?requis=1` et mémorise l'adresse demandée. Après connexion, l'utilisateur retourne où il voulait aller ; cette adresse est revérifiée par `Chemins.cheminSur` avant la redirection.

8.3 Politique de sécurité du contenu

Listing 8.1. Politique appliquée à toutes les réponses

```
default-src 'self'; img-src 'self' data: https:;
style-src 'self' 'unsafe-inline'; script-src 'self';
frame-ancestors 'none'; form-action 'self'; base-uri 'self'
```

`script-src 'self'` est le cœur : seuls les scripts servis par l'application s'exécutent. `frame-ancestors 'none'` interdit l'inclusion de l'application dans un cadre d'un autre site (détournement de clic) ; `form-action 'self'` empêche un formulaire injecté d'envoyer ses données ailleurs. `style-src` garde `'unsafe-inline'` parce que JSF et les composants produisent des attributs `style` ; c'est un compromis assumé, le risque d'un style injecté étant bien moindre que celui d'un script.

8.4 Contrôle d'accès

Un utilisateur qui n'a pas le rôle requis (par exemple un client devant une page d'administration) reçoit une réponse 403 affichée dans la page d'erreur de l'application, pas la page technique de Tomcat.

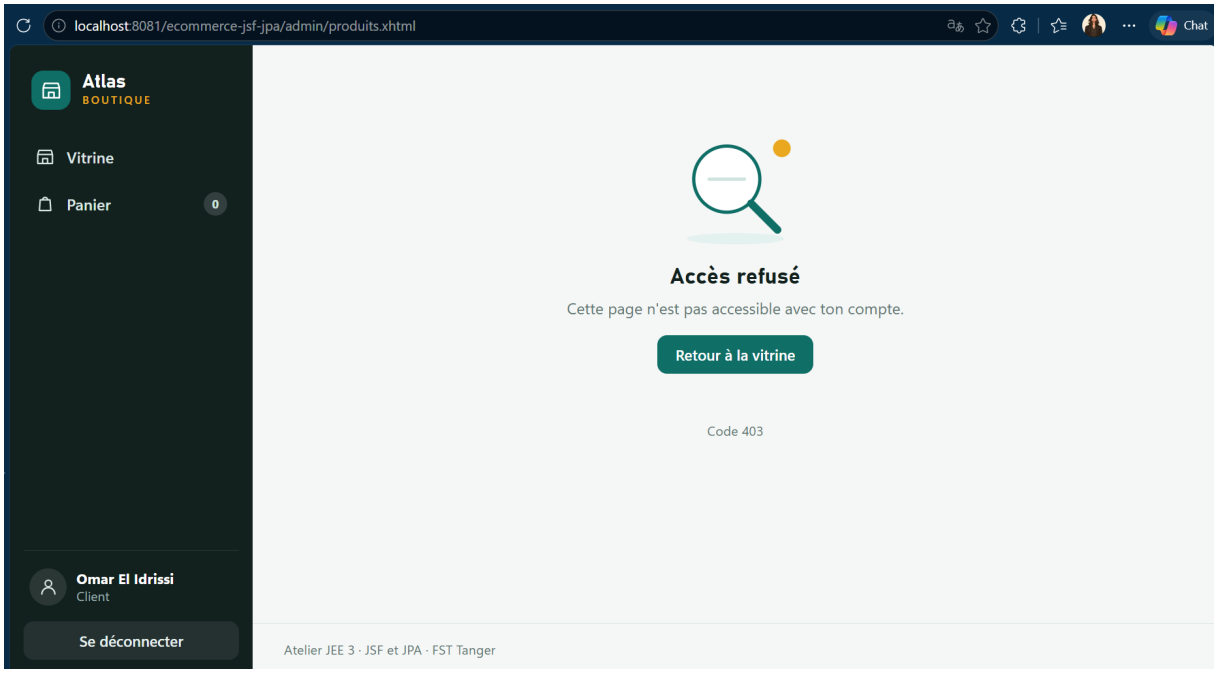


Figure 8.1. Accès refusé : un client demande une page d'administration

CHAPITRE 9

Tests

9.1 Stratégie

Trois niveaux de vérification se complètent. Les contrôles automatisés couvrent le métier, les utilitaires, la persistance, les parties pures de la couche web et la structure statique des pages ; ils s'exécutent par une seule commande et affichent un total. Le parcours manuel vérifie ce qu'aucun test sans serveur ne peut voir (rendu, cycle de vie JSF, redirections). Les captures de ce rapport en sont la trace.

9.2 Contrôles automatisés

La classe `Verifications` enchaîne six lots, sans bibliothèque de test : une petite classe `Checks` compte les réussites et les échecs. Le résultat attendu est 215 réussites et aucun échec.

Table 9.1. Lots de contrôles

Lot	Ce qu'il vérifie
`PanierCheck`	Règles R1 à R6 du panier : bornes de quantité, stock, fusion de lignes, total en `BigDecimal`.
`ServiceChecks`	Services métier : validation des formulaires, messages d'erreur.
`UtilChecks`	Hachage PBKDF2, argent, limiteur de tentatives.
`JpaChecks`	Persistance réelle sur MySQL : contraintes refusées, verrouillage, héritage (nécessite la base).
`WebChecks`	Parties pures de la couche web : redirections sûres, jeton CSRF, politique CSP, adresses de la vitrine, aides d'affichage, page d'erreur.
`VuesChecks`	Analyse statique des pages Facelets (section 9.4).

9.3 Tests de la couche web

`WebChecks` montre l'intérêt de sortir la logique des pages et des beans dans des méthodes pures. Quelques exemples :

— `Chemins.cheminSur` refuse `//site-externe`, les URL absolues, l'antislash et un saut de ligne (injection d'en-tête) ; le chemin interne est accepté.

— `VitrineBean.construireUrl` encode la recherche en UTF-8 (`café noir` → `caf%C3%A9+noir`), omet la page 1 et neutralise un `&page=9` saisi dans la recherche.

— `SecurityFilter.egal` compare les jetons sans fuite par le temps et refuse un jeton absent ou de longueur différente.

— `Affichage.prix` formate avec deux décimales, une virgule et une espace insécable avant « MAD ».

9.4 Analyse statique des vues

Une erreur dans une page Facelets n'apparaît normalement qu'au chargement de la page. `VuesChecks` la détecte avant, sans serveur :

1. chaque page, gabarit et composant est un document XML bien formé ;
2. aucun `h:commandLink`, aucun `f:ajax`, aucun attribut `on*` (cohérence avec la CSP) ;
3. chaque `h:form` contient `t:csrf` ;
4. chaque expression `bean.membre` désigne un bean existant et un accesseur ou une méthode qui existe.

La dernière règle remplace une partie de ce qu'un EL mal écrit ferait découvrir en pleine démonstration. Elle a déjà trouvé un faux positif instructif : une expression EL laissée dans un commentaire XML d'un composant. Facelets évalue l'EL même dans les commentaires ; le commentaire a été reformulé.

9.5 Parcours manuel

Le tableau 9.2 liste les quinze scénarios déroulés dans le navigateur. Chaque capture porte la barre d'adresse et le titre de l'onglet, afin que la page ouverte soit identifiable.

Table 9.2. Scénarios du parcours manuel

N°	Scénario et résultat attendu	Figure
1	Ouvrir la vitrine : cartes, pastilles de catégories.	7.1
2	Rechercher, puis filtrer : l'adresse porte `q` et `categorie`, le compteur change.	7.2
3	Ouvrir un produit : prix, stock, « Se connecter pour acheter ».	7.3
4	Connexion avec un mauvais mot de passe : message rouge, champ vidé.	7.4
5	Connexion correcte : « Bienvenue », badge du panier.	—
6	Ouvrir le panier : lignes, quantités, total.	7.6
7	Dépasser le stock du produit `INF-003` : refus, panier inchangé ; produit à stock nul : bouton désactivé.	7.7
8	Ajouter un produit déjà présent : fusion de la ligne ; `F5` ne rejoue pas le formulaire.	—
9	Contrôler la base : les lignes correspondent à l'écran.	9.1
10	Se déconnecter : retour à la vitrine avec message.	—
11	Connexion administrateur : arrivée sur la liste des produits.	7.8
12	Nouveau produit, formulaire vide : erreurs sous les champs.	7.9
13	Inscription avec un mot de passe trop court : erreurs, mots de passe vidés.	7.5
14	Client devant `admin/produits.xhtml` : page 403.	8.1
15	Non connecté devant `panier.xhtml` : redirection vers la connexion, puis retour au panier.	—

Le scénario 8 vérifie le motif Post/Redirect/Get : après l'ajout, la page affichée est le résultat d'une redirection (`GET`), donc un rechargement n'envoie rien. Le scénario 7 se fait par ajouts répétés : un navigateur bloque de toute façon une saisie au-dessus du maximum d'un champ numérique, c'est pourquoi la règle est aussi vérifiée côté serveur.

```
PS C:\dev\ecommerce-jsf-jpa> mysql -u root -p ecommerce_jsf -e "
SELECT i.email, p.reference, l.quantite, l.prix_unitaire FROM li
gne_panier l JOIN panier pa ON pa.id=l.panier_id JOIN internaute
 i ON i.panier_id=pa.id JOIN produit p ON p.id=l.produit_id ORDE
R BY i.email;"
Enter password: ******************
+--------------------+-----------+----------+---------------+
| email              | reference | quantite | prix_unitaire |
+--------------------+-----------+----------+---------------+
| yasmine@example.com | INF-001   |        2 |        549.00 |
| yasmine@example.com | INF-003   |        3 |        899.00 |
+--------------------+-----------+----------+---------------+
```

Figure 9.1. Contrôle en base : lignes du panier après les manipulations

CHAPITRE 10

Incidents et diagnostics

Ce chapitre rassemble les problèmes rencontrés et la manière dont chacun a été diagnostiqué. Ils sont instructifs parce que, dans trois cas sur quatre, le symptôme visible ne désignait pas la cause.

10.1 Erreur 500 après une connexion réussie

Symptôme. Le formulaire de connexion accepte les identifiants, mais la page suivante est l'erreur 500 de l'application ; en cliquant sur le panier ou la vitrine, on retombe sur la connexion avec « Connecte-toi pour continuer ».

Diagnostic. Le symptôme secondaire (retour à la connexion) laisse croire à un défaut d'authentification. Il n'en est rien : la trace de Tomcat montre une exception au moment de la redirection.

Listing 10.1. Trace de Tomcat (abrégée)

```
SEVERE: Servlet.service() for servlet [FacesServlet] in context with path
[/ecommerce-jsf-jpa] threw exception [javax/xml/bind/DatatypeConverter]
java.lang.ClassNotFoundException: javax.xml.bind.DatatypeConverter
  at com.sun.faces.util.ByteArrayGuardAESCTR.encrypt(ByteArrayGuardAESCTR.java:139)
  at com.sun.faces.context.flash.ELFlash$PreviousNextFlashInfoManager.encode(ELFlash.java
    :1489)
  at com.sun.faces.context.flash.ELFlash.doLastPhaseActions(ELFlash.java:722)
  at com.sun.faces.context.ExternalContextImpl.redirect(ExternalContextImpl.java:656)
  at com.sun.faces.application.NavigationHandlerImpl.handleNavigation(NavigationHandlerImpl.
    java:256)
```

Cause. Mojarra 2.2.x chiffre le cookie du *flash* (qui transporte le message « Bienvenue » à travers la redirection) avec `javax.xml.bind.DatatypeConverter`. Cette classe appartenait au JDK jusqu'à Java 8 ; elle a été retirée avec Java 11. Sous Java 17, la première redirection qui conserve un message échoue donc, ici au moment de la navigation qui suit une connexion réussie. Le retour à la page de connexion s'explique, selon mon hypothèse, par l'identifiant de session renouvelé juste avant (`changeSessionId`) : la réponse 500 ne l'ayant pas transmis au navigateur, les requêtes suivantes ne reconnaissent plus l'utilisateur. Cette seconde partie est déduite et non observée directement ; la trace, elle, est sans ambiguïté.

Correction. Ajouter `javax.xml.bind:jaxb-api:2.3.1` (extrait 3.1). Aucun changement de code.

Leçon. Une exception à la redirection peut ressembler à un problème d'authentification. On lit d'abord le journal du serveur, pas la page.

10.2 Faux positifs d'analyse statique dans Eclipse

Symptôme. Eclipse (outils web) signale des erreurs dans `vitrine.xhtml` et `erreur.xhtml` : « Could not coerce choice argument to boolean » et « Could not make numeric conversion », pour des expressions qui fonctionnent à l'exécution.

Cause. L'analyseur d'EL ne connaît pas le type des variables d'un composant ni celui d'un attribut de requête (`requestScope['javax.servlet.error.status_code']`) ; il ne peut pas vérifier une expression conditionnelle qui les mélange.

Correction. Les décisions d'affichage sont passées dans Java (`Affichage`, `VitrineBean`) : la page appelle une méthode dont le type de retour est connu. La logique est de ce fait testable (chapitre 9). Les avertissements restants (espaces de noms de passage, attributs SVG) sont sans effet et ne sont pas contournés.

10.3 Projet de l'atelier précédent dans l'espace de travail

Symptôme. L'onglet *Problems* d'Eclipse affiche une erreur dans `panier.jsp`, fichier qui n'existe pas dans le nouveau projet.

Diagnostic. Une recherche de `*.jsp` dans le dossier du projet ne renvoie rien, et `git status` du projet de l'atelier 2 est propre : l'erreur vient du projet de l'atelier 2, ouvert dans le même espace de travail.

Correction. Fermer l'autre projet, le retirer du serveur et filtrer la vue *Problems* sur le projet courant. Le projet lui-même n'a pas été modifié. La phrase « No persistence.xml found in project » relève d'une autre cause : la facette JPA d'Eclipse, inutile ici puisque la persistance est configurée à la main, a été décochée.

10.4 Classes Java 17 illisibles par EclipseLink

Reconduit de l'atelier 2 et traité à la section 3.5 : sans `release=11`, aucune entité n'est découverte et la base reste vide.

CHAPITRE 11

Gestion de version

11.1 Principes

Le dépôt `jee-ecommerce-jsf-jpa` est public sur GitHub. Il suit les règles de l'atelier 2 : commits atomiques qui compilent, messages au format *Conventional Commits* [14], et aucun secret versionné (`db.properties` est ignoré ; seul `db.properties.example` est suivi).

11.2 Historique

Le tableau 11.1 donne la séquence. Chaque commit correspond à une couche ; le rapport est ajouté en dernier.

Table 11.1. Séquence de commits

Étape	Message
Projet	`chore: initialise Maven project, database script and configuration template`
Persistance	`feat(persistence): add JPA entities, utilities and schema tool`
Services	`feat(service): add business services and JPA implementations`
Web	`feat(web): add JSF managed beans, filters and configuration`
Vues	`feat(views): add Facelets layout, composite components and pages`
Tests	`test: add business, security and Facelets static checks`
Rapport	`docs(report): add LaTeX report, figures and PDF`

CHAPITRE 12

Perspectives

— **La commande.** Transformer un panier en commande (adresse, paiement, historique) reste volontairement hors périmètre, comme à l'atelier 2. Les services sont prêts à l'accueillir : un `CommandeService` et un `CommandeBean` s'ajouteraient sans modifier l'existant.

— **Ajax.** Un panier qui se met à jour sans rechargement est naturel avec `f:ajax`. Il suppose de choisir entre assouplir la CSP et fournir le code JavaScript depuis le serveur : c'est une décision de sécurité avant d'être une décision de confort.

— **Bibliothèque de composants.** PrimeFaces ou un équivalent apporterait tables paginées et boîtes de dialogue. J'ai choisi JSF pur pour comprendre ce que ces bibliothèques cachent ; cela devient légitime une fois acquis.

— **Jakarta EE.** Le passage à Tomcat 10 ou à un serveur d'applications impose l'espace de noms `jakarta.*` et JSF 3 ou plus. La séparation en couches limite le changement aux imports et aux descripteurs.

— **Limiteur de tentatives.** Le `LoginGuard` est en mémoire : il ne survit pas à un redémarrage et ne se partage pas entre serveurs. Un déploiement en grappe demanderait un stockage commun.

CHAPITRE 13

Conclusion

L'atelier 3 a transformé une application MVC2 en application JSF sans toucher à une seule classe de la couche métier ni de la persistance. Cette observation est la plus importante du travail : la séparation en couches de l'atelier 2 a rempli son rôle, et le coût d'un changement de technologie web s'est limité à quinze classes et seize pages.

Trois idées en ressortent. D'abord, JSF déplace la réflexion du *chemin d'une requête* vers le *cycle de vie d'une vue* ; les portées (`@ViewScoped`, jamais `@SessionScoped` ici) et le motif Post/Redirect/Get demandent une attention constante. Ensuite, une politique de sécurité stricte contraint le choix des composants : renoncer à `h:commandLink` et à `f:ajax` est un arbitrage, pas une limite. Enfin, sortir la logique d'affichage dans des méthodes Java rend les vues plus simples, les tests possibles et les analyseurs statiques silencieux.

Les limites sont connues : la commande n'est pas traitée, le limiteur de connexions est local à un serveur, et Tomcat, MySQL et Maven n'ont été vérifiés que sur ma machine, les tests automatisés couvrant ce qui s'exécute sans serveur. Le rapport rend compte de ces deux niveaux de vérification et des incidents qui les ont séparés.

Bibliographie

[1] Oracle / JCP. *JSR 344 : JavaServer Faces 2.2*. https://jcp.org/en/jsr/detail?id=344

[2] Eclipse Foundation. *Mojarra, implémentation de JSF*. https://eclipse-ee4j.github.io/mojarra/

[3] Oracle / JCP. *JSR 365 : Contexts and Dependency Injection for Java 2.0*. https://jcp.org/en/jsr/detail?id=365

[4] Oracle / JCP. *JSR 338 : Java Persistence 2.2*. https://jcp.org/en/jsr/detail?id=338

[5] Oracle / JCP. *JSR 369 : Java Servlet 4.0*. https://jcp.org/en/jsr/detail?id=369

[6] Eclipse Foundation. *EclipseLink Documentation (2.7)*. https://www.eclipse.org/eclipselink/documentation/

[7] Red Hat. *Weld, implémentation de référence de CDI*. https://weld.cdi-spec.org/

[8] Apache Software Foundation. *Apache Tomcat 9 Documentation*. https://tomcat.apache.org/tomcat-9.0-doc/

[9] Oracle. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/

[10] OWASP. *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/

[11] OWASP. *Unvalidated Redirects and Forwards Cheat Sheet*. https://cheatsheetseries.owasp.org/

[12] W3C. *Content Security Policy Level 3*. https://www.w3.org/TR/CSP3/

[13] *Post/Redirect/Get*, motif de conception web. https://en.wikipedia.org/wiki/Post/Redirect/Get

[14] *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/

[15] M. Fowler. *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002 (Front Controller, Page Controller, Single Table Inheritance).

CHAPITRE A

Annexes

A.1 Arborescence du projet

Listing A.1. Arborescence simplifiée

```
jee-ecommerce-jsf-jpa/
  pom.xml, README.md, .gitignore
  db/create-database.sql
  docs/diagramme-classes.md, docs/rapport/
  src/main/java/ma/fstt/
    entities/  exception/  util/  persistence/  service/  jpa/   (atelier 2)
    web/       15 classes : filtres, beans, UserSession, Affichage
  src/main/resources/
    META-INF/persistence.xml, META-INF/beans.xml, db.properties.example
  src/main/webapp/
    WEB-INF/web.xml, faces-config.xml, templates/layout.xhtml
    resources/components/  csrf, icone, visuel, vide, champ
    *.xhtml (8 pages) et admin/*.xhtml (4 pages)
    assets/css/app.css, assets/js/app.js
  src/test/java/ma/fstt/  Verifications + 7 lots de contrôles
```

A.2 Comptes de démonstration

Table A.1. Comptes créés par `SchemaTool`

Profil	Adresse	Mot de passe
Administrateur	`admin@example.com`	`Admin#2026`
Client	`yasmine@example.com`	`Client#2026`

Un second client, `omar@example.com`, sert au scénario de refus d'accès. Ces comptes n'existent que dans la base locale de démonstration.

A.3 Diagrammes Mermaid

Le fichier `docs/diagramme-classes.md` contient deux diagrammes que GitHub affiche directement. Le premier est le modèle du domaine ; le second la couche JSF.

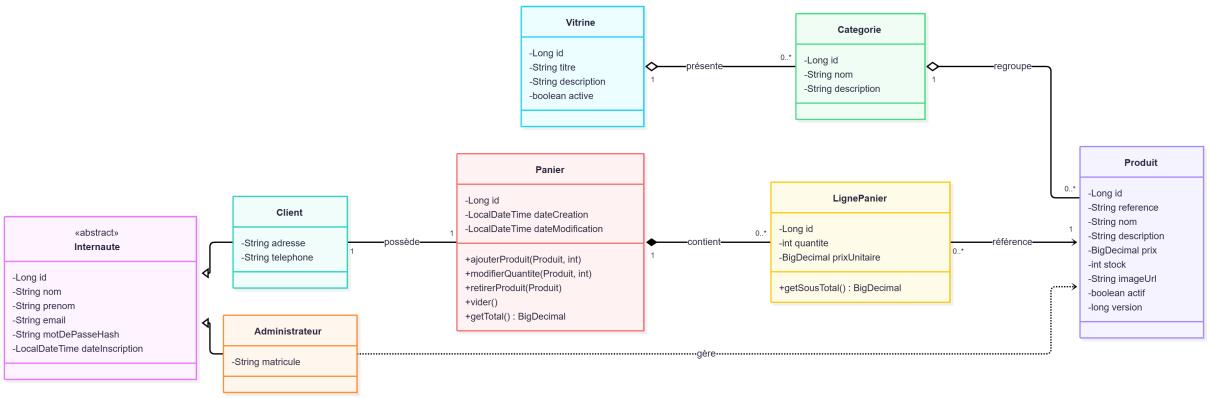


Figure A.1. Diagramme Mermaid du modèle métier

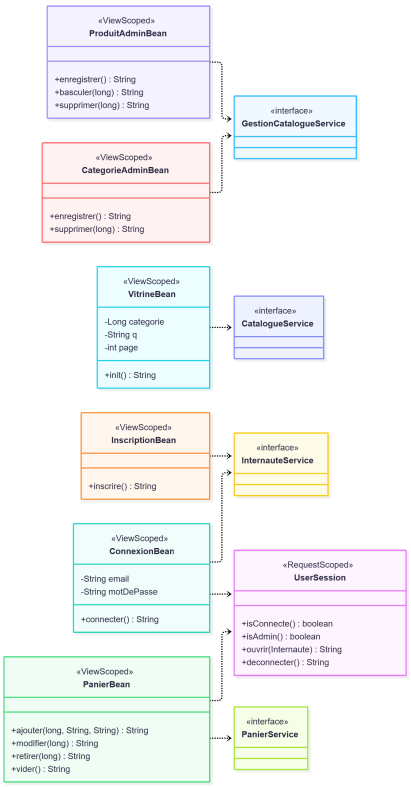


Figure A.2. Diagramme Mermaid de la couche JSF